Object-Oriented Python From Scratch

No assumed background

August 2026

Contents

Object-Oriented Python From Scratch

No assumed OOP background. By the end you'll understand why the ICF Level 1 skeleton is shaped the way it is — because that's the only OOP you need for this assessment.

Type every example into a REPL (`python3` in a terminal). Reading this without typing is roughly half as useful.

---

1. The problem OOP solves

Say you're tracking users, each with a name and a score. Without classes:

```
names  = ["ann", "bob"]
scores = [10, 20]
```

Two lists you must keep in sync. Delete a user and you must remember to delete from both. Add a third property (email) and every function that touches users needs updating. This falls apart quickly.

A dict is better:

```
users = {"ann": {"score": 10}, "bob": {"score": 20}}
users["ann"]["score"] += 5
```

But nothing stops you writing `users["ann"]["scr"]` — a typo silently creates a new field. And the *rules* about users (a score can't go below zero; a user expires after 30 days) live scattered across whatever functions happen to touch the dict.

A class bundles the data and the rules that govern it into one place. That's the whole idea. Everything else is syntax.

---

2. Your first class

```
class User:
    def __init__(self, name, score):
        self.name = name
        self.score = score
```

Type that, then:

```
u = User("ann", 10)
u.name          # 'ann'
u.score         # 10
u.score = 15
u.score         # 15
```

Vocabulary, all for the same few things:

- `User` is a **class** — a blueprint.
- `u` is an **instance** or **object** — one thing built from the blueprint.
- `name` and `score` are **attributes** — the data on an instance.
- `User("ann", 10)` **instantiates**. Note: no `new` keyword; you call the class like a function.

What `__init__` is

`__init__` runs automatically when you create an instance. Its job is to set up the starting attributes. The double underscores mean "Python calls this for you" — you never call `__init__` yourself.

What `self` is

`self` is the instance being worked on. When you write `User("ann", 10)`, Python creates an empty object and passes it in as `self`. So `self.name = name` means "store this argument on *this particular object*."

The rule that trips up everyone coming from other languages: **`self` is written explicitly in every method definition, but never passed at the call site.**

```
class User:
    def __init__(self, name):
        self.name = name

    def greet(self):              # self required here
        return f"hi {self.name}"

u = User("ann")
u.greet()                         # not passed here — Python does it
```

`u.greet()` is really `User.greet(u)` — try typing that, it works.

Try it: add a `score` attribute and a method `bump(self, amount)` that adds to it and returns the new score.

---

3. Methods — the rules live with the data

A **method** is just a function defined inside a class. Its value is that it can read and change the instance's attributes via `self`.

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            return False                    # the rule lives here
        self.balance += amount
        return True

    def withdraw(self, amount):
        if amount > self.balance:
            return False                    # and here
        self.balance -= amount
        return True
```

```
a = Account("ann", 100)
a.deposit(50)      # True,  balance 150
a.withdraw(500)    # False, balance unchanged
a.balance          # 150
```

Now the "you can't overdraw" rule exists in exactly one place. Anyone using `Account` gets it for free. That's the payoff.

Note `balance=0` — a **default argument**. `Account("bob")` works and starts at zero.

Try it: add a `transfer(self, other, amount)` method that withdraws from `self` and deposits into `other`, returning `False` if the withdrawal fails. (Yes, a method can take another instance as an argument.)

---

4. Instance attributes vs. class attributes

This one causes real bugs.

```
class Bad:
    items = []                      # CLASS attribute — ONE list, shared

    def add(self, x):
        self.items.append(x)

a, b = Bad(), Bad()
a.add(1)
b.items                             # [1]  ← b sees a's data!
```

```
class Good:
    def __init__(self):
        self.items = []             # INSTANCE attribute — one per object

    def add(self, x):
        self.items.append(x)
```

```
a, b = Good(), Good()
a.add(1)
b.items                          # []   ← correct
```

The rule: anything mutable (list, dict, set) goes in `__init__`. Class attributes are fine for constants (`MAX_SIZE = 100`) and nothing else until you know why you want otherwise.

---

5. `__repr__` — making objects printable

By default, printing an object is useless:

```
u = User("ann")
print(u)          # <__main__.User object at 0x7f9c...>
```

Add `__repr__` and debugging gets dramatically easier:

```
class User:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __repr__(self):
        return f"User({self.name!r}, score={self.score})"

print(User("ann", 10))            # User('ann', score=10)
print([User("ann", 10)])          # [User('ann', score=10)]  ← in lists too
```

In a timed assessment where you're printing state to find a bug, this pays for its two lines many times over.

`__repr__` is a **dunder** (double-underscore) method — one Python calls automatically in specific situations. You've now seen two: `__init__` on creation, `__repr__` on printing.

---

6. Properties — computed attributes

Sometimes a value should be derived, not stored:

```
class Record:
    def __init__(self, created_at, ttl=None):
        self.created_at = created_at
        self.ttl = ttl

    @property
    def expires_at(self):
        if self.ttl is None:
            return None
        return self.created_at + self.ttl
```

```
r = Record(5, 10)
r.expires_at       # 15   ← NO parentheses
r.ttl = 20
r.expires_at       # 25   ← recomputed automatically
```

`@property` is a **decorator** — a line starting with `@` above a definition that modifies how it behaves. You don't need to understand decorators in general; you just need to recognize this one.

Why it matters for your test: when you read an unfamiliar class, you must notice whether a member is a property or a method, because calling a property (`r.expires_at()`) produces a confusing `TypeError` far from the real mistake. Check for the `@property` line above the definition.

Try it: add a method `alive_at(self, t)` returning `True` if the record hasn't expired at time `t`. (Method, not property — it takes an argument.)

---

7. Dataclasses — less typing for data holders

Writing `__init__` by hand gets tedious when a class is mostly data. A **dataclass** generates it for you.

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class Record:
    key: str                              # required
    value: int
    created_at: int = 0                   # default
    ttl: Optional[int] = None
    log: list = field(default_factory=list)

    @property
    def expires_at(self):
        return None if self.ttl is None else self.created_at + self.ttl

    def alive_at(self, t):
        return self.ttl is None or t < self.expires_at
```

```
r = Record("k", 5)
r                       # Record(key='k', value=5, created_at=0, ...)  free __repr__
r2 = Record("k", 5, created_at=3, ttl=10)
r2.alive_at(12)         # True
r2.alive_at(13)         # False
```

You get `__init__`, `__repr__`, and `__eq__` written for you. The `key: str` syntax is a **type hint** — documentation that Python doesn't enforce, but dataclasses use it to know what the fields are.

Three rules that will bite you:

1. **Mutable defaults need `field(default_factory=list)`.** Writing `log: list = []` raises an error — for the same shared-state reason as section 4.
2. **Fields with defaults must come after fields without them.**
3. Dataclass instances are **unhashable** by default, so they can't go in a `set`. Use `@dataclass(frozen=True)` if you need that.

When to use which: dataclass for things that are mostly data (a record, a request, a file entry); a regular class for things that mostly *do* something (the database, the scheduler, the engine).

---

8. Inheritance (know it, rarely need it)

One class can build on another:

```
class Animal:
    def __init__(self, name):
        self.name = name
```

```
    def speak(self):
        raise NotImplementedError        # subclasses must define this

class Dog(Animal):                       # Dog inherits from Animal
    def speak(self):                     # override
        return "woof"

class Cat(Animal):
    def __init__(self, name, indoor):
        super().__init__(name)           # run Animal's __init__ first
        self.indoor = indoor

    def speak(self):
        return "meow"
```

```
Dog("rex").name        # 'rex'  — inherited from Animal
Dog("rex").speak()     # 'woof'
isinstance(Dog("rex"), Animal)     # True
```

Three things to recognize:

- **`class Dog(Animal)`** — Dog is a kind of Animal.
- **`super().__init__(...)`** — call the parent's version.
- **`raise NotImplementedError`** — a stub saying "subclass must fill this in." Your mock assessments use exactly this for the methods you implement.

You will probably not *need* inheritance on this assessment. You need to recognize it when reading given code.

---

9. Putting it together: why the ICF skeleton looks like that

Everything above exists to make this readable:

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class Field:                             # §7: dataclass for pure data
    value: int
    created_at: int = 0
    ttl: Optional[int] = None

    def alive_at(self, t):               # §3: the rule lives with the data
        return self.ttl is None or t < self.created_at + self.ttl


class Database:                          # §2: a regular class — it *does* things
    def __init__(self):
        self.store = {}                  # §4: mutable state in __init__
        self.log = []

    # public API — the signatures tests call
    def set(self, key, field, value):
        return self._set(0, key, field, value)

    def set_at(self, timestamp, key, field, value, ttl=None):
        return self._set(timestamp, key, field, value, ttl)
```

```
    # one internal implementation, shared            §3: rules in one place
    def _set(self, t, key, field, value, ttl=None):
        self.log.append((t, "set", (key, field, value, ttl)))
        self.store.setdefault(key, {})[field] = Field(value, t, ttl)
```

Read that again with the sections in mind. Every choice traces back to something above: a dataclass because `Field` is data; `alive_at` on `Field` because the expiry rule belongs with the thing that expires; state in `__init__` because it's mutable; two public methods delegating to one internal because duplicated logic drifts.

The leading underscore in `_set` is convention meaning "internal, not part of the public API." Python doesn't enforce it — there is no `private` keyword.

---

10. Exercises (do these — 25 minutes)

A. Inventory — Write a class `Inventory` with `__init__` creating an empty dict, then `add(item, qty)`, `remove(item, qty)` returning `False` if there isn't enough, and `count(item)` returning 0 for unknown items. Add `__repr__`.

B. Convert to a dataclass — Write `@dataclass class Item` with fields `name: str`, `qty: int = 0`, `tags: list` (defaulting to empty — remember the trap), plus a property `is_empty`.

C. The delegation pattern — Give `Inventory` an `add_at(timestamp, item, qty)` that records the timestamp in a log, and make the original `add` call it with timestamp 0. Both must work. This is the single most important pattern for your assessment.

D. Read someone else's class — Open `mock_oa_kv/solution/database_solution.py` and answer in writing: which members are dataclasses vs. regular classes? Which are properties vs. methods? Where does `rollback` get the information it needs? Why does `_live_fields` exist instead of the filtering being repeated in four places?

Then check drills `c9` and `c10` in `day1_drills.py`.

What to skip

You do not need, for this assessment: multiple inheritance, metaclasses, abstract base classes, `__slots__`, class methods and static methods, operator overloading beyond `__repr__`, or descriptors. If you meet them in someone else's code, they're readable enough to interpret in context.